PUC Minas

Testes de Software

Detecção de Bad Smells e Refatoração Segura

Aluno: Gabriel Reis Lebron de Oliveira
Matrícula: 855324
Repositório: https://github.com/GabrielReiiss/bad-smells-js-refactoring

Belo Horizonte, novembro de 2026

1. Análise de Smells

1.1 Método Longo

Onde aparece: no método `generateReport` (linhas 11 a 68), com 58 linhas. Sozinho, ele monta o cabeçalho, filtra os itens por perfil, marca a prioridade, formata as linhas, soma o total e monta o rodapé.

Problema para a manutenção: para mudar uma regra é preciso entender o método inteiro. Os comentários “Seção do Cabeçalho”, “Seção do Corpo” e “Seção do Rodapé” marcam blocos que deveriam ser métodos próprios.

Problema para os testes: o filtro e a formatação não podem ser testados separadamente. Todo teste precisa gerar o relatório inteiro e inspecionar o texto final, e uma falha não indica qual parte quebrou.

1.2 Condicionais Repetidas e Aninhadas

Onde aparece: a verificação de `CSV` ou `HTML` se repete em 4 blocos (linhas 16, 35, 46 e 58), e o perfil é verificado nas linhas 28 e 43. No pior caso, há 4 níveis de aninhamento (for, perfil, valor e formato).

Problema para a manutenção: um novo formato exige alterar os 4 blocos, e um novo perfil exige copiar todo o bloco de formatos.

Problema para os testes: os caminhos se multiplicam, pois cada formato se combina com cada perfil e cada faixa de valor.

1.3 Código Duplicado

Onde aparece: a linha do CSV é idêntica nas linhas 36 e 47, a do HTML é quase igual nas linhas 40 e 50, e `total += item.value` aparece 4 vezes (linhas 37, 41, 48 e 51).

Problema para a manutenção: uma correção precisa ser repetida em cada cópia, e as cópias acabam ficando diferentes. Foi o que aconteceu aqui. A linha HTML do Admin gerava `<tr >`, com um espaço sobrando, e a do User não.

Problema para os testes: testar uma cópia não garante nada sobre a outra. O erro no `<tr >` só apareceu no cenário do Admin, embora o cenário do User verificasse a mesma linha.

Também foram observados os números mágicos `1000` e `500` (linhas 30 e 45), a alteração de `item.priority` em um objeto recebido por parâmetro (linha 32) e o campo `this.db`, que nunca é usado (linha 3).

2. Relatório da Ferramenta

O `eslint-plugin-sonarjs` 0.21.0 foi executado com o ESLint 8.57.1 e o `.eslintrc.json` do enunciado, que substituiu o arquivo do repositório e alterou o limite de complexidade cognitiva de 5 para 15. O resultado de `npx eslint src/` antes da refatoração foi:

```
C:\Users\Gabriel Reis\Documents\Projetos\bad-smells-js-refactoring\src\ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                          sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Regra do ESLint	Linha	Smell correspondente (análise manual)
`sonarjs/cognitive-complexity` (erro)	11	Método Longo e Condicionais Aninhadas, complexidade 27, para um limite de 15
`sonarjs/no-collapsible-if` (erro)	43	Condicionais Aninhadas, `if` que pode ser unido ao `else if`

A ferramenta acrescentou o que a leitura manual não oferece. A regra `cognitive-complexity` transformou a impressão de “código complicado” em uma medida objetiva, que pesa mais os trechos aninhados e define quando a refatoração atingiu o objetivo. A regra `no-collapsible-if` apontou o `if` da linha 43, que havia passado despercebido na análise manual.

Por outro lado, a duplicação e os números mágicos só foram encontrados na leitura manual. A regra `no-identical-functions` compara funções inteiras, e aqui a repetição está dentro de um único método. A ferramenta complementa a revisão manual, mas não a substitui.

3. Processo de Refatoração

O smell escolhido foi o das **Condicionais Repetidas e Aninhadas**, o mais crítico, pois é o principal responsável pela complexidade de 27 e o que obrigava a repetir as linhas do relatório em cada combinação de perfil e formato.

Antes de refatorar, foram necessários dois ajustes para obter a rede de segurança que o enunciado pede. O `npm test` não executava, pois o Jest exige a opção `--experimental-vm-modules` em projetos com `"type": "module"`, então ela foi adicionada ao script `test` do `package.json`. Em seguida, 1 dos 5 testes falhou por causa do `<tr >` citado na seção 1.3. Como os testes não podem ser alterados, as duas linhas responsáveis foram corrigidas no arquivo original, e a suíte chegou a 5 de 5 testes aprovados.

As técnicas aplicadas em `src/ReportGenerator.refactored.js` foram:

- **Replace Conditional with Polymorphism.** Cada formato virou um objeto com os mesmos métodos (header, row e footer). O formato é escolhido uma única vez no `Map formatters`, e um novo formato só exige um novo objeto.
- **Decompose Conditional.** As regras de perfil viraram funções com nomes claros (isAdmin, markPriority e canView).
- **Extract Method.** A seleção de itens e a soma do total foram para `selectVisibleItems` e `sumValues`, e `generateReport` apenas coordena as etapas. Os números `1000` e `500` também viraram constantes com nome.

Antes: src/ReportGenerator.js

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    // Admins veem todos os itens
    if (item.value > 1000) {
      // Lógica bônus para admins
      item.priority = true;
    }

    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    // Users comuns só veem itens de valor baixo
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
        total += item.value;
      }
    }
  }
}
```

Depois: src/ReportGenerator.refactored.js

```
function selectVisibleItems(items, user) {
  const visibleItems = [];
  for (const item of items) {
    markPriority(item, user);
    if (canView(item, user)) {
      visibleItems.push(item);
    }
  }
  return visibleItems;
}

const csvFormatter = {
  header() {
    return 'ID,NOME,VALOR,USUARIO\n';
  },
  row(item, user) {
    return `${item.id},${item.name},${item.value},${user.name}\n`;
  },
  footer(total) {
    return `\nTotal,,\n${total},,\n`;
  },
};

generateReport(reportType, user, items) {
  const formatter = formatters.get(reportType) ?? emptyFormatter;
  const header = formatter.header(user);
  const visibleItems = selectVisibleItems(items, user);
  const rows = visibleItems.map((item) => formatter.row(item, user)).join('');
  const footer = formatter.footer(sumValues(visibleItems));

  return (header + rows + footer).trim();
}
```

A refatoração foi feita em 3 passos (constantes, condições e formatos). O `npm test` foi executado depois de cada passo e não houve falhas. A cópia do teste, `tests/ReportGenerator.refactored.test.js`, difere do original apenas na linha do `import`. O resultado final foi:

```
PASS tests/ReportGenerator.test.js
PASS tests/ReportGenerator.refactored.test.js

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
```

Indicador	Antes	Depois
Erros do ESLint no arquivo	2	0
Complexidade cognitiva de `generateReport`	27	1
Linhas de `generateReport`	58	9
Testes aprovados	5 de 5	5 de 5

4. Conclusão

Os testes foram o que tornou esta refatoração segura. Como a suíte verifica o resultado, e não a implementação, o código pôde ser totalmente reorganizado, e cada passo foi confirmado em segundos pelo `npm test`. A suíte também mostrou seu valor antes da refatoração, ao revelar o erro do `<tr >` em um código entregue como funcional. Ela tem limites, pois os valores exatos de 500 e 1000 não são testados, o que exigiu atenção manual.

A redução dos Bad Smells melhorou a qualidade geral do software. O método principal agora se lê em poucas linhas, cada regra fica em um único lugar e um novo formato não exige alterar o código que já funciona. Testes, análise estática e revisão manual se complementam. O ESLint mede o problema, a leitura encontra o que a ferramenta não vê, e os testes garantem que a correção não mudou o comportamento.